Lecture 9: Computational Cameras

Linear Inverse Problems & Deconvolution

Instructor: Katie Bouman

TAs: Brandon Zhao, Brady Bhalla, Zachary Huang

Course announcements

- HW 3 due today, HW 4 on Canvas today
- Ombuds – extra ½ late days
- Project ideas on the course website later today.
  - Sample project ideas posted on Canvas, but you can come up with your own ideas.
  - Proposal due May 14
  - 2-4 page report and a 10 minute presentation

Overview of today’s lecture

- Sources of blur
- Deconvolution
- Blind deconvolution

Slide credits

Many of these slides/images were borrowed/adapted from:

- Ioannis (Yannis) Gkioulekas (CMU)
- William Freeman (MIT)
- Fredo Durand (MIT).
- Gordon Wetzstein (Stanford)
- Daniel Zoran (Google Brain)

Why are our images blurry?

3 that we have discussed in class, and 1 we haven’t yet discussed

Why are our images blurry?

- Camera shake
- Scene motion
- Depth defocus
- Lens imperfections

Why are our images blurry?

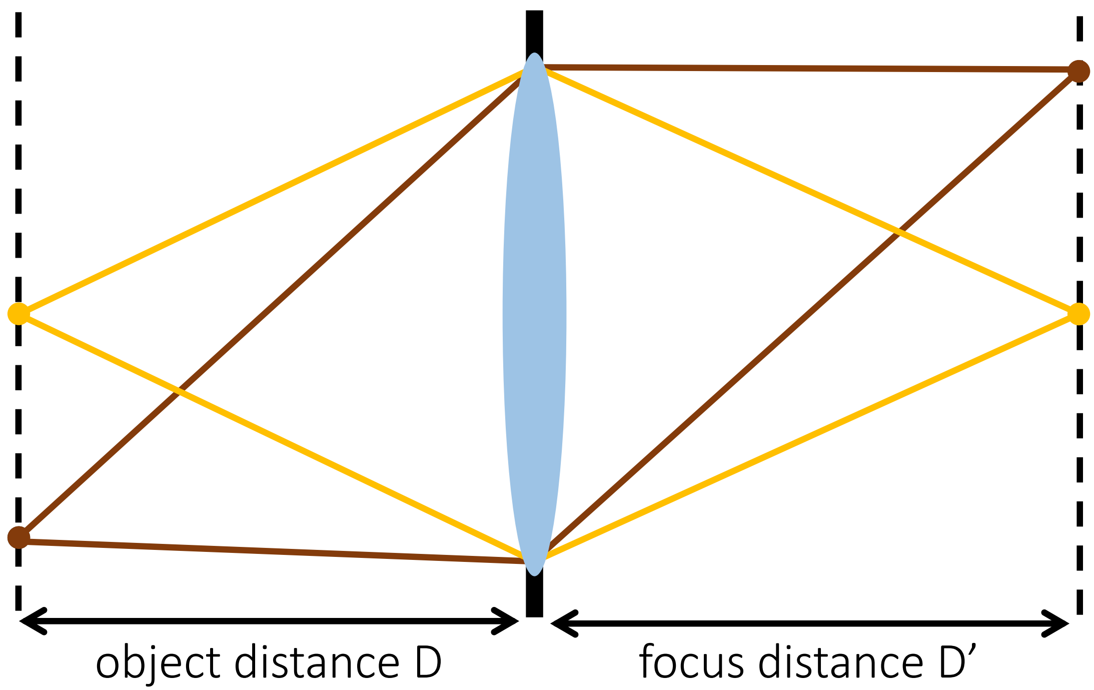
- Camera shake
- Scene motion
- Depth defocus
- Lens imperfections

Why are our images blurry?

- Camera shake
- Scene motion
- Depth defocus
- Lens imperfections

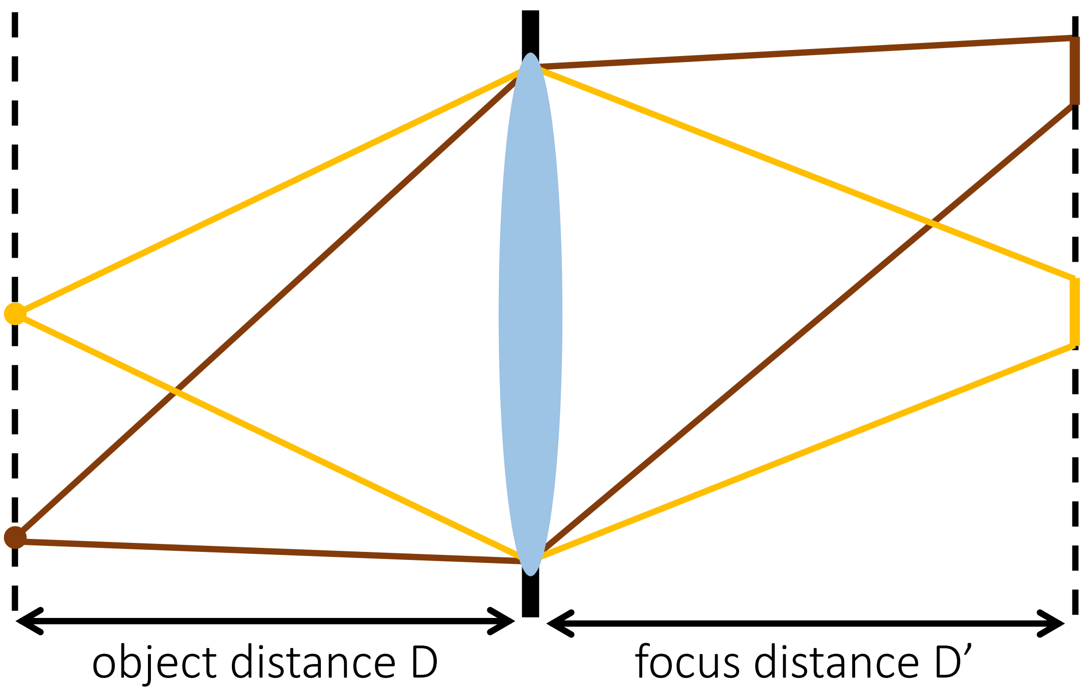
Lens imperfections

- Ideal lens: An point maps to a point at a certain plane.



Lens imperfections

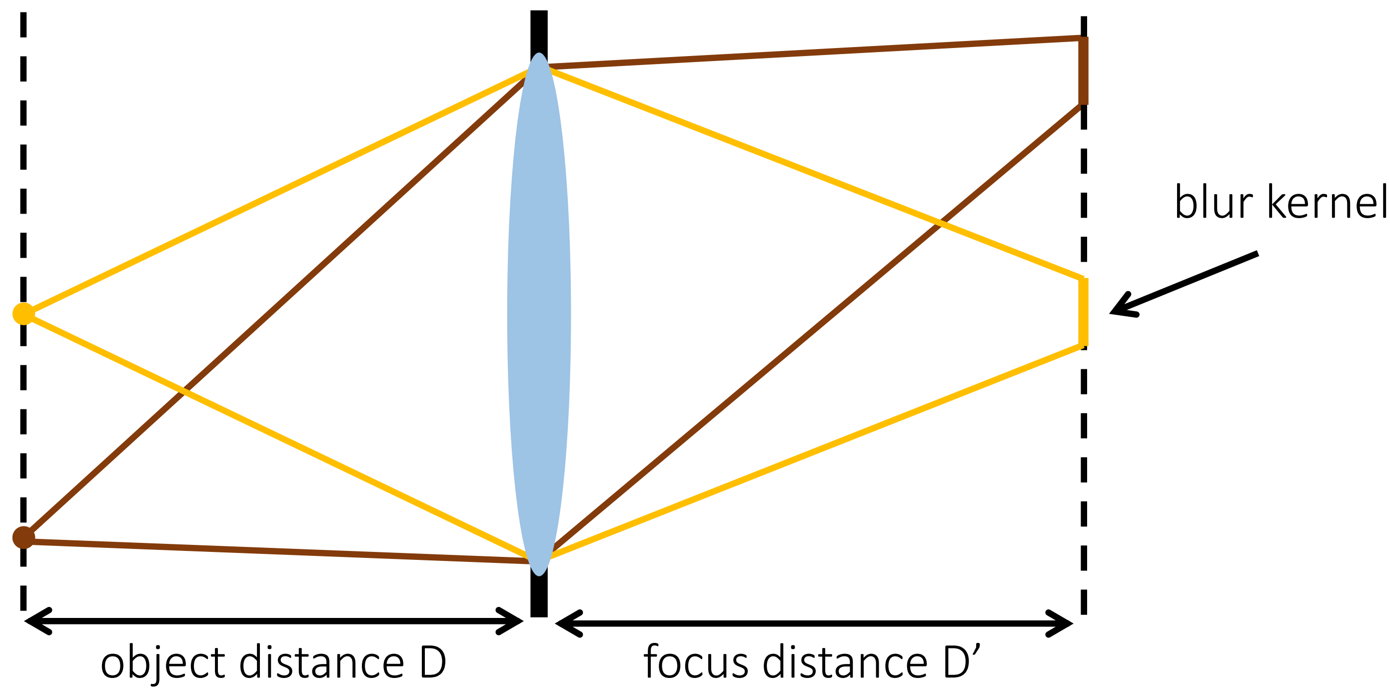
- Ideal lens: An point maps to a point at a certain plane.
- Real lens: A point maps to a circle that has non-zero minimum radius among all planes.



What is the effect of this on the images we capture?

Lens imperfections

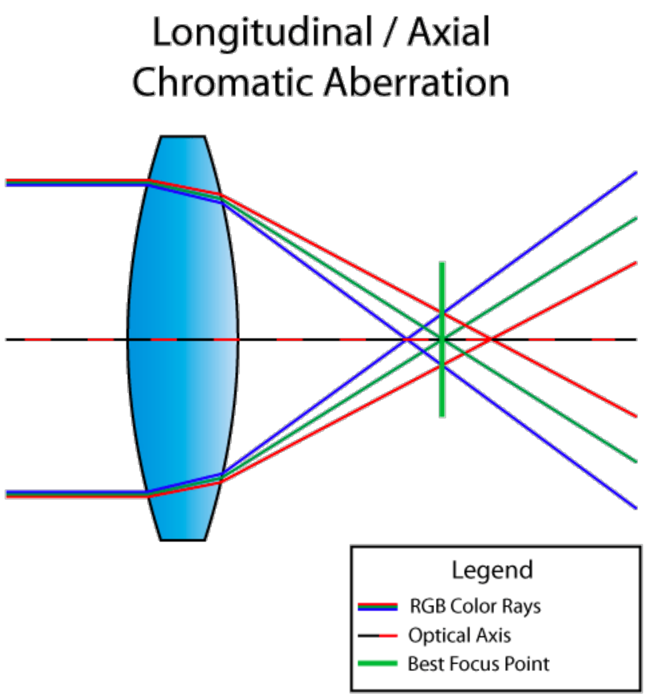
- Ideal lens: An point maps to a point at a certain plane.
- Real lens: A point maps to a circle that has non-zero minimum radius among all planes.

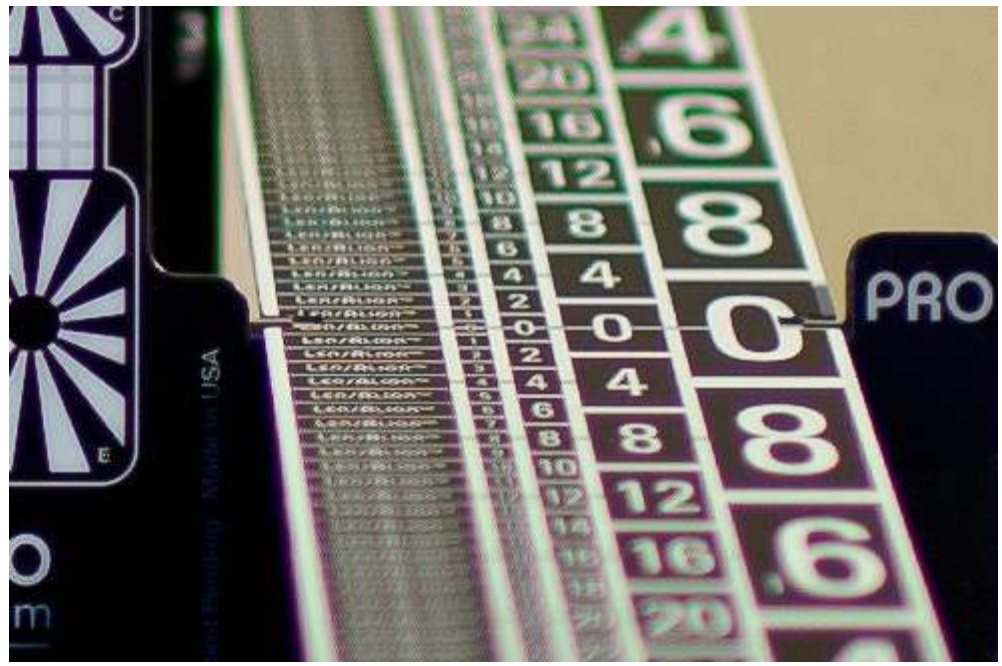


What are common lens imperfections?

- Aberrations
- Diffraction

Chromatic aberration

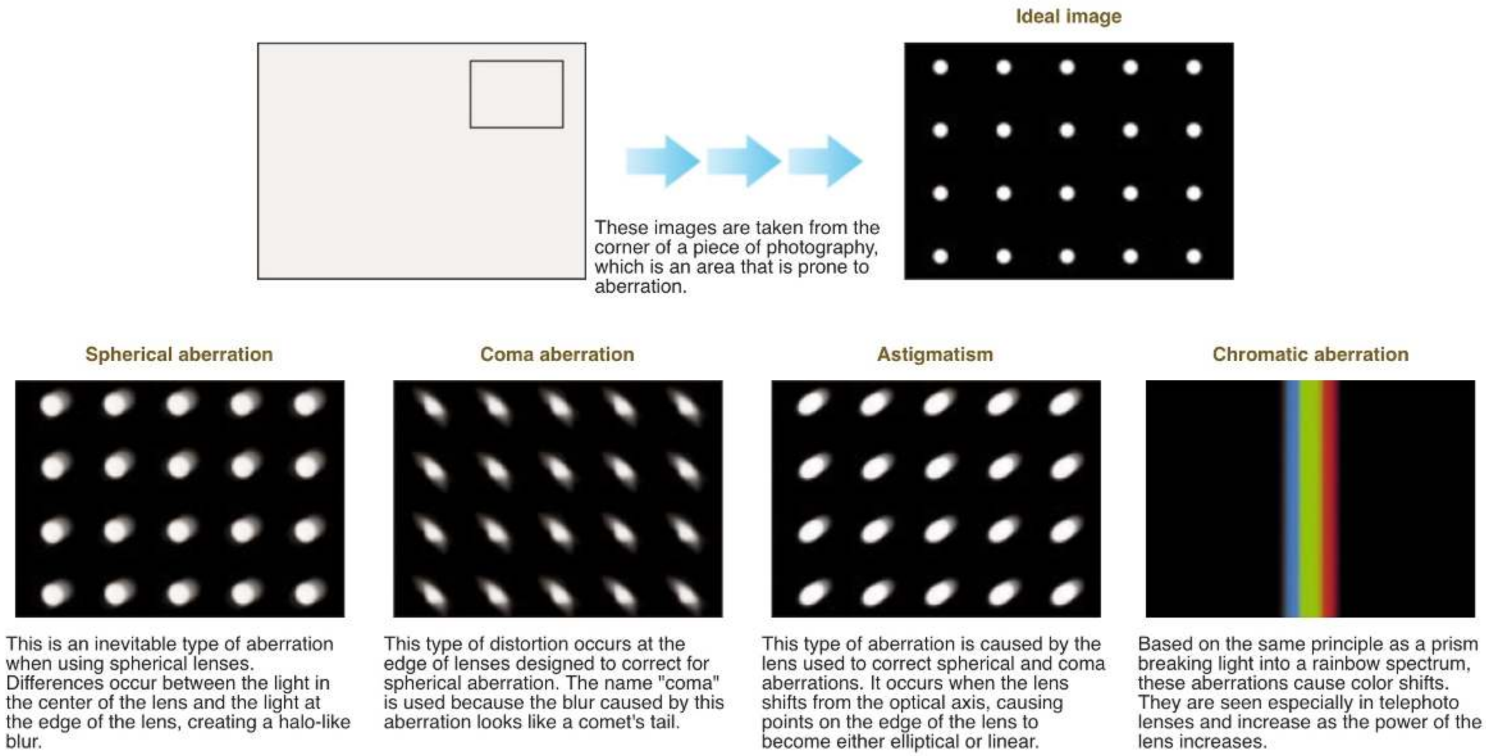




photographylife.com

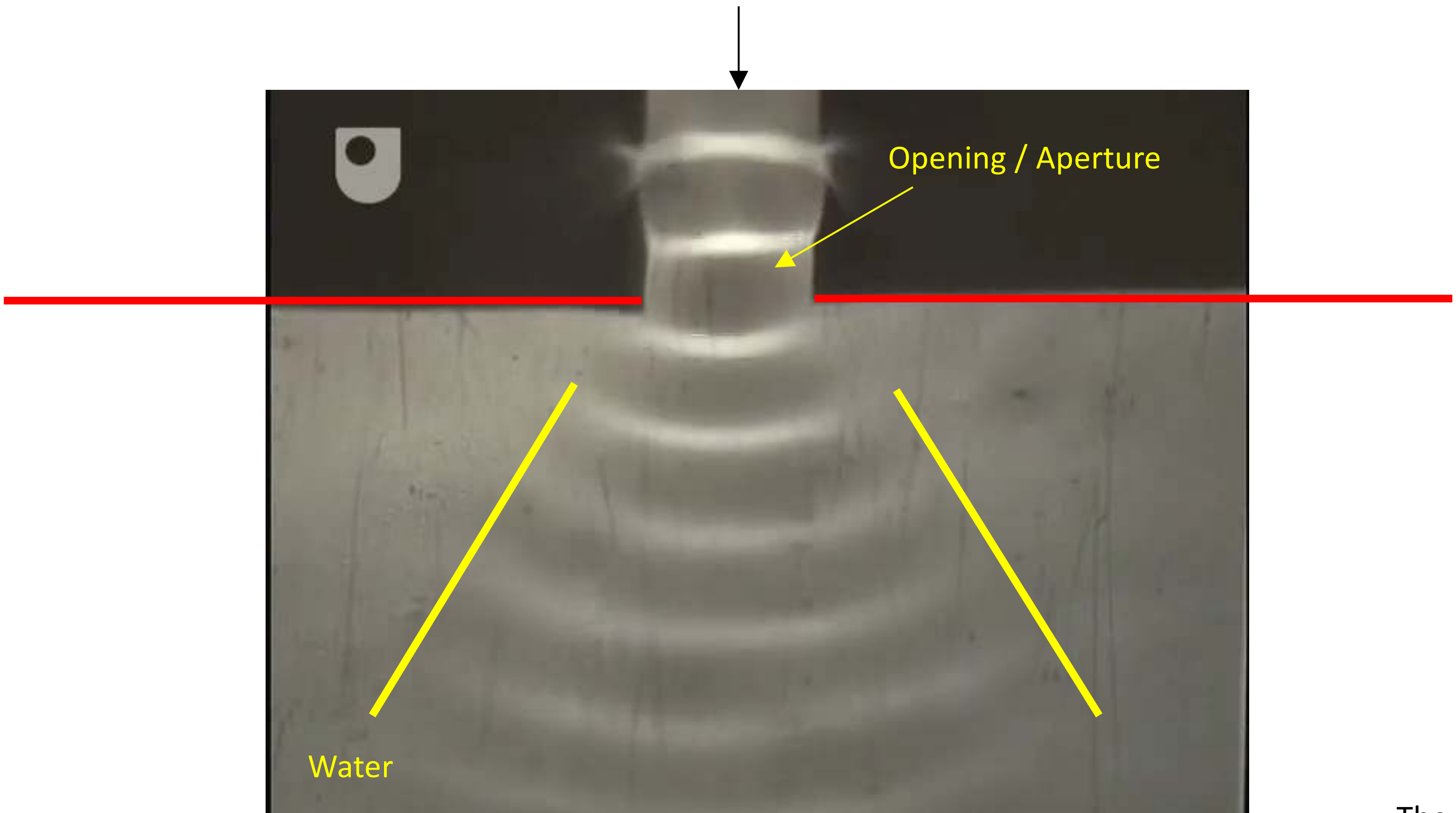
Optical aberrations

Note: Oblique aberrations are often not shift-invariant blur, so we do not consider them here



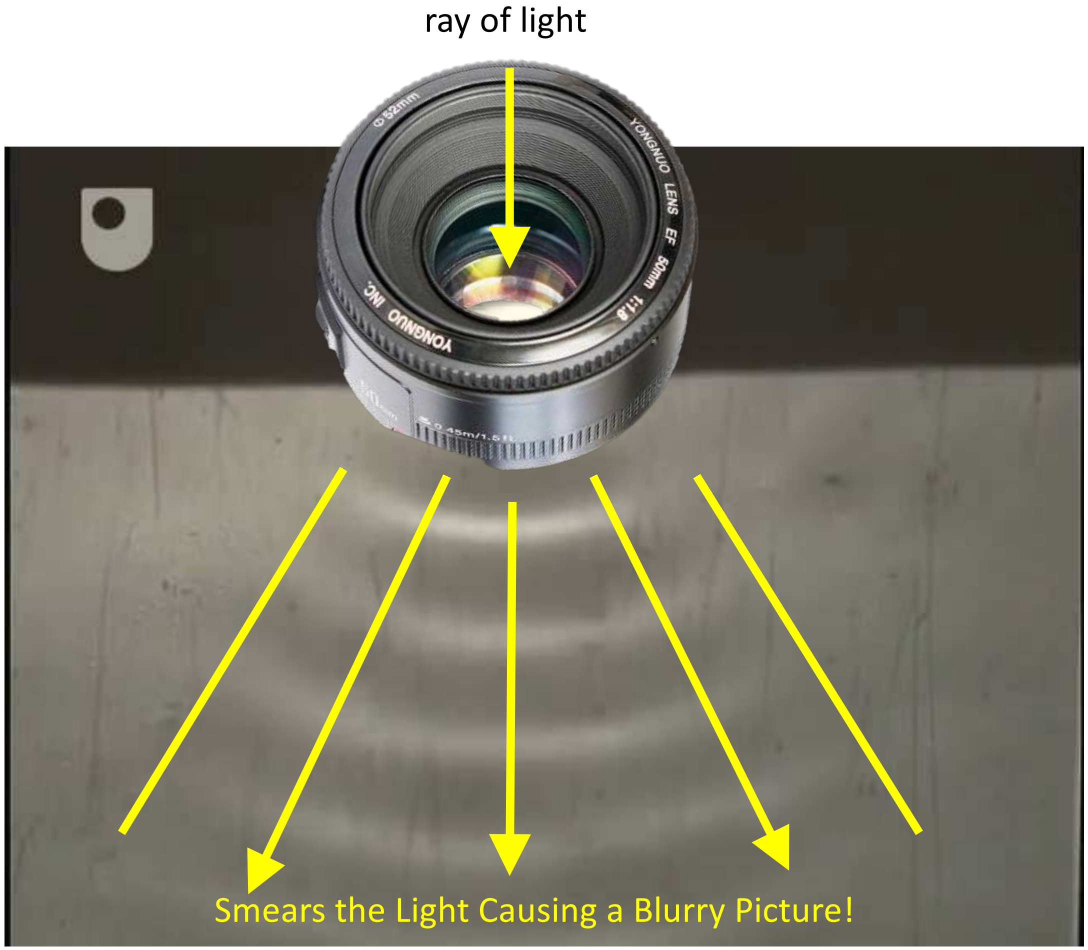
panasonic.com

Diffraction



The Open University

Diffraction



The Open University

Diffraction in everyday cameras

f/5.6

f/22

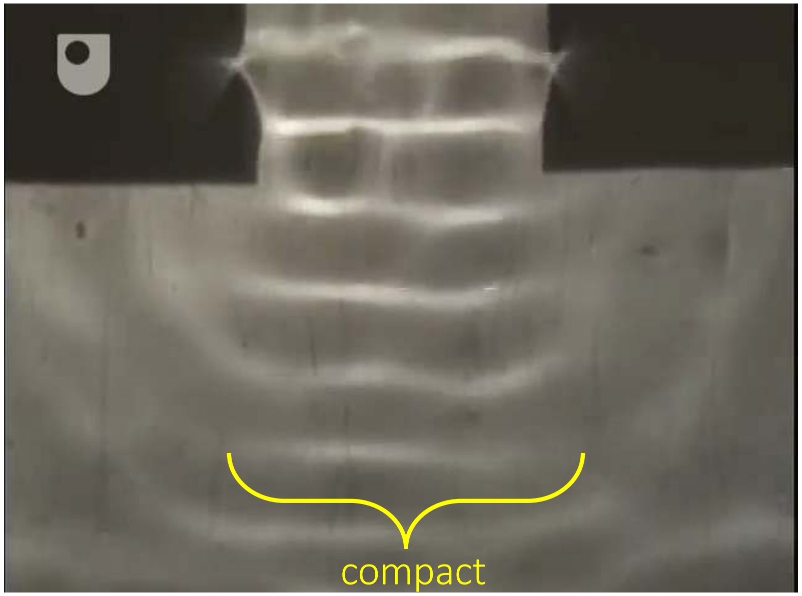
f/45

Decreasing Camera Aperture →

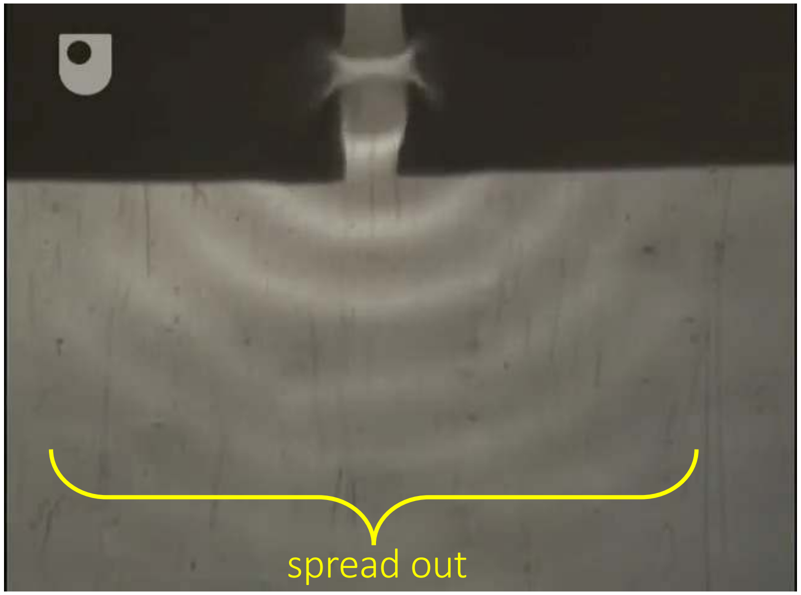
Diffraction in everyday cameras

Diffraction vs. aperture size

Large Aperture/Slit



Small Aperture/Slit



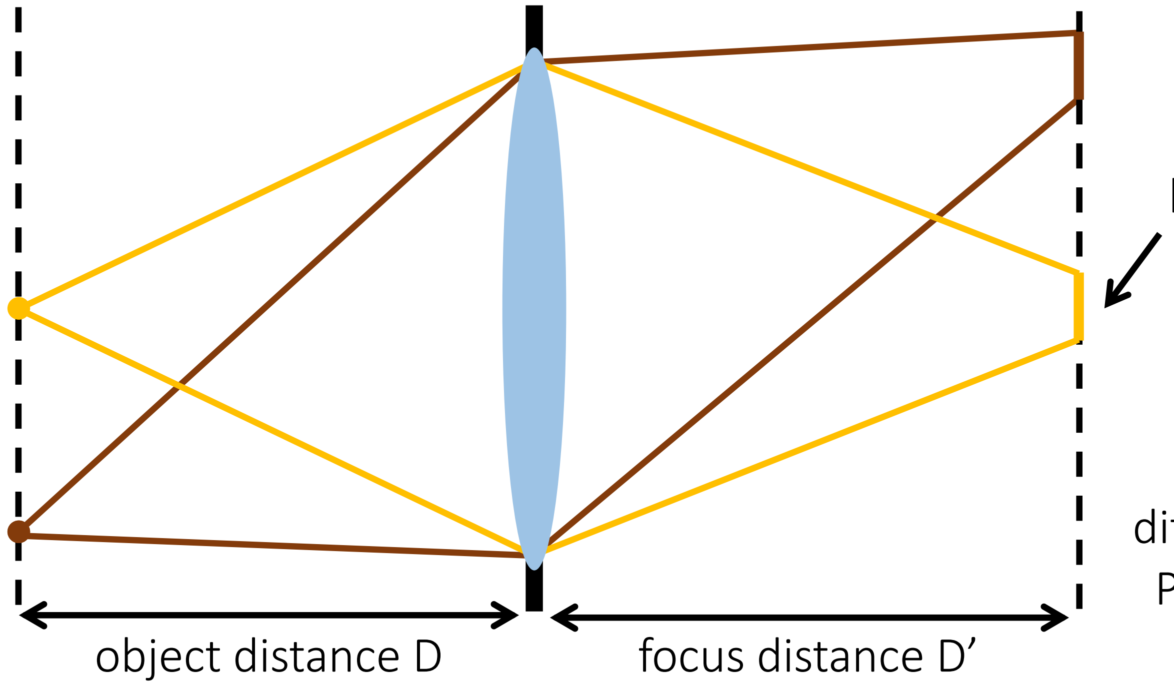
the smaller the aperture the blurrier the picture

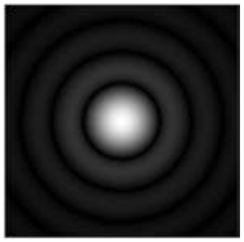
The Open University

Lens as an optical low-pass filter

Point spread function (PSF): The blur kernel of a lens.

- "Diffraction-limited" PSF: No aberrations, only diffraction. Determined by aperture shape.



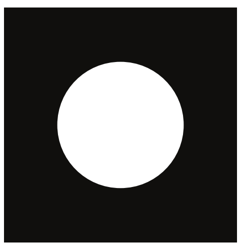


diffraction-limited PSF of a circular aperture

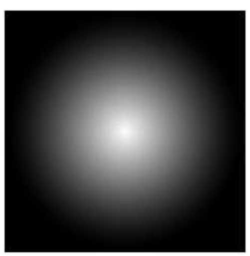
Lens as an optical low-pass filter

Point spread function (PSF): The blur kernel of a lens.

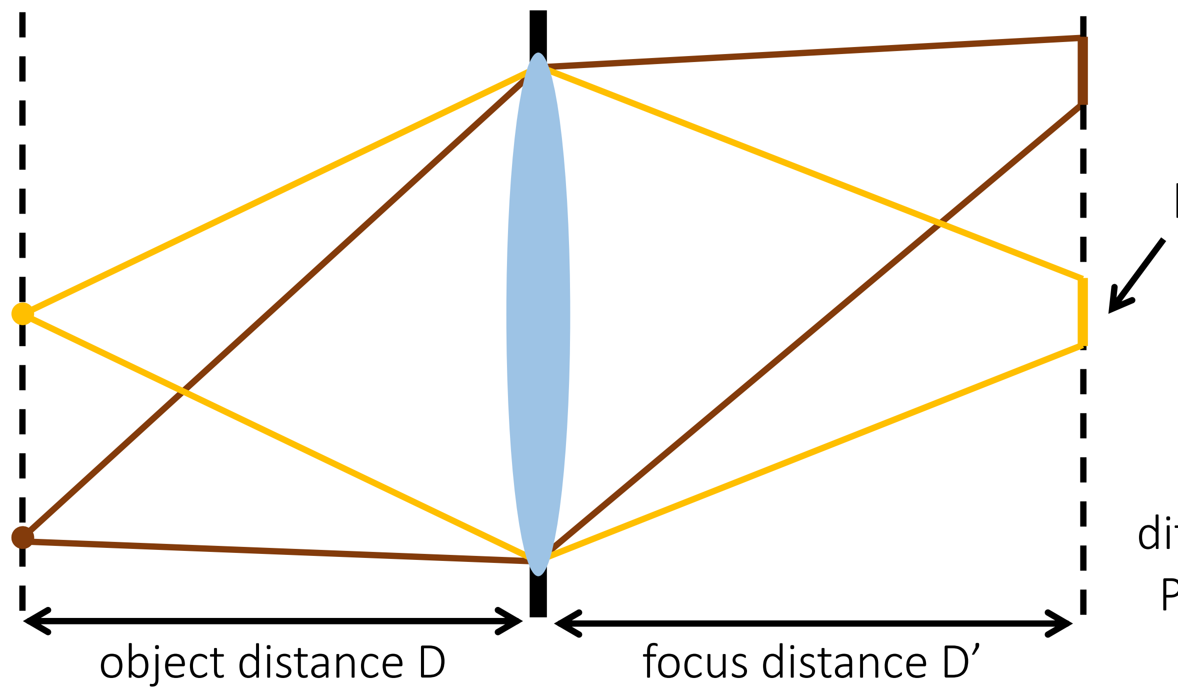
- "Diffraction-limited" PSF: No aberrations, only diffraction. Determined by aperture shape.

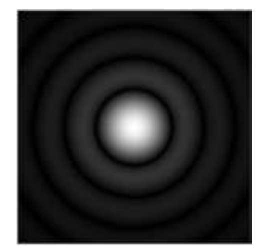


Aperture



OTF of a circular aperture





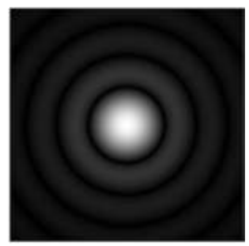
diffraction-limited PSF of a circular aperture

Optical transfer function (OTF): The Fourier transform of the PSF, related to aperture shape.

Lens as an optical low-pass filter

image from a perfect lens

$*$



imperfect lens PSF

$=$

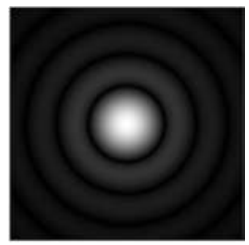
image from imperfect lens

$$x \quad * \quad c \quad = \quad b$$

Lens as an optical low-pass filter

If we know c and b, can we recover x?

*



=

image from a perfect lens	imperfect lens PSF	image from imperfect lens

$$x \quad * \quad c \quad = \quad b$$

Deconvolution

$$x \quad * \quad c \quad = \quad b$$

If we know c and b, can we recover x?

Deconvolution

$$x \quad * \quad c \quad = \quad b$$

Reminder: convolution is multiplication in Fourier domain:

$$F(x) \cdot F(c) = F(b)$$

If we know c and b, can we recover x?

Deconvolution

$$x \quad * \quad c \quad = \quad b$$

Reminder: convolution is multiplication in Fourier domain:

$$F(x) \;\cdot\; F(c) \;=\; F(b)$$

Deconvolution is division in Fourier domain:

$$F(x_{est}) = F(b) \;\backslash\; F(c)$$

After division, just do inverse Fourier transform:

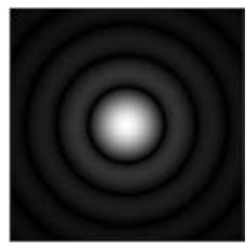
$$x_{est} \;=\; F^{-1}\left(F(b) \backslash F(c) \right)$$

Deconvolution result

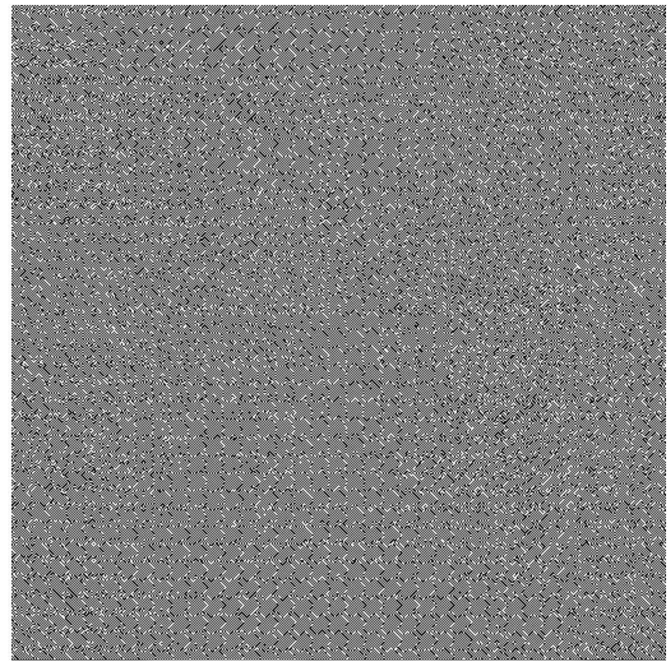
Why is the result so bad?

- Event a tiny amount of noise can cause awful results - example for Gaussian of σ = 0.05

$*$



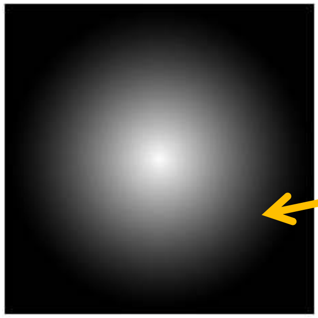
$^{-1}$ $=$



$$b \ast c^{-1} = x_{est}$$

Deconvolution

- The OTF (Fourier of PSF) is a low-pass filter
  optical transfer function



zeros at high frequencies

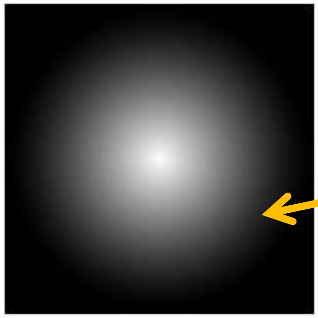
- The measured signal includes noise

$$b = c * x + n$$

noise term

Deconvolution

- The OTF (Fourier of PSF) is a low-pass filter



zeros at high frequencies

- The measured signal includes noise

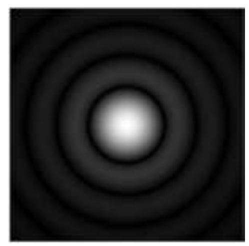
$$b = c * x + n$$

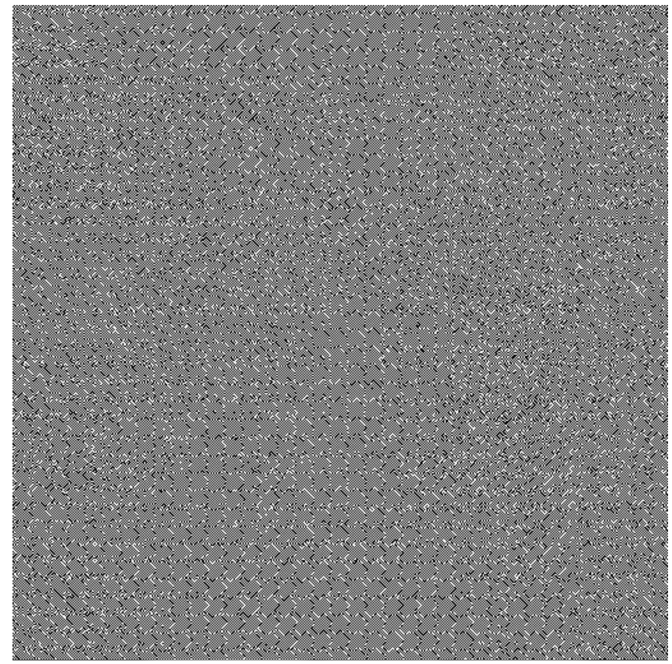
noise term

- When we divide by zero, we amplify the high frequency noise

Naïve deconvolution

- Event a tiny amount of noise can cause awful results - example for Gaussian of σ = 0.05

$*$ ()$^{-1}$ $=$



$$b \quad * \quad c^{-1} \quad = \quad x_{est}$$

Wiener Deconvolution

Apply inverse kernel and do not divide by zero:

$$x_{est} = F^{-1}\left(\boxed{\frac{|F(c)|^2}{|F(c)|^2 + 1/SNR(\omega)}} \cdot \frac{F(b)}{F(c)} \right)$$

noise-dependent damping factor

- Requires noise of signal-to-noise ratio at each frequency

$$SNR(\omega) = \frac{\text{Expected signal squared at } \omega}{\text{Zero-mean noise variance at } \omega}$$

Wiener Deconvolution

Apply inverse kernel and do not divide by zero:

$$x_{est} = F^{-1}\left(\boxed{\frac{|F(c)|^2}{|F(c)|^2 + 1/SNR(\omega)}} \cdot \frac{F(b)}{F(c)}\right)$$

noise-dependent damping factor

- As noise increases for a particular frequency, does this multiplier increase or decrease?

increase

decrease

Wiener Deconvolution

Apply inverse kernel and do not divide by zero:

$$x_{est} = F^{-1}\left(\boxed{\frac{|F(c)|^2}{|F(c)|^2 + 1/SNR(\omega)}} \cdot \frac{F(b)}{F(c)}\right)$$

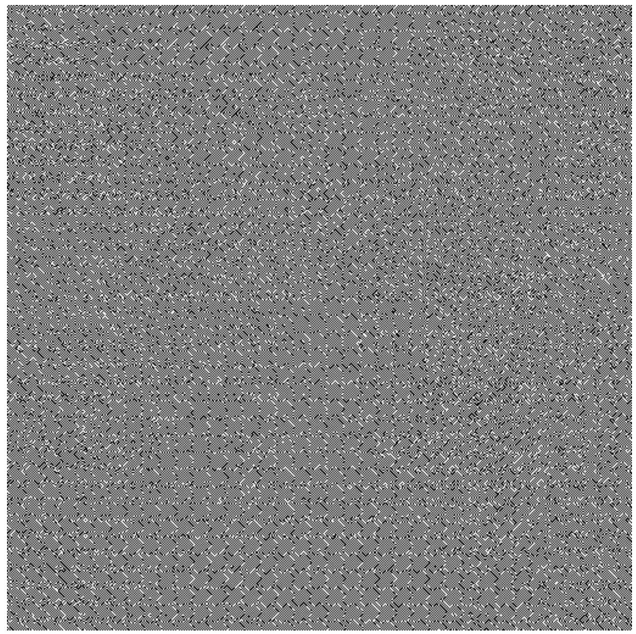
noise-dependent damping factor

Intuitively:

- When SNR is high (low or no noise), just divide by kernel.
- When SNR is low (high noise), just set to zero.

Deconvolution comparisons

blurred



naïve deconvolution

Wiener deconvolution

Deconvolution comparisons

σ = 0.01

σ = 0.05

σ = 0.1

Derivation

Sensing model:

$$b = c * x + n$$

Noise $\mathbf{n}$ is assumed to be zero-mean and independent of signal $\mathbf{x}$.

Derivation

Sensing model:

$$b = c * x + n$$

Noise $\mathbf{n}$ is assumed to be zero-mean and independent of signal $\mathbf{x}$.

Fourier transform:

$$B = C \cdot X + N$$

lowercase is for spatial domain, UPPERCASE is for frequency domain

Derivation

Sensing model:

$$b = c * x + n$$

Noise n is assumed to be zero-mean and independent of signal x.

Fourier transform:

$$B = C \cdot X + N$$

Convolution becomes multiplication.

Problem statement: Find function $\mathrm{H}(\omega)$ that minimizes *expected* error *in Fourier domain*.

$$\min_{H} E[\|X - HB\|^2]$$

Derivation

Problem statement: Find function $\mathrm{H}(\omega)$ that minimizes *expected* error *in Fourier domain*.

$$\min_H E[\|X - H(CX + N)\|^2]$$

Replace B and re-arrange loss:

$$\min_H E[\|(1 - HC)X - HN\|^2]$$

Expand the squares:

$$\min_H \;\; \|1 - HC\|^2 E[\|X\|^2] - 2H(1 - HC)E[XN] + \|H\|^2 E[\|N\|^2]$$

Derivation

When handling the cross terms:

- Can I write the following?

$$E[XN] = E[X]E[N]$$

 YES

 No

Derivation

When handling the cross terms:

- Can I write the following?

$$E[XN] = E[X]E[N]$$

Yes, because X and N are assumed independent.

- What is this expectation product equal to?

Derivation

When handling the cross terms:

- Can I write the following?

$$E[XN] = E[X]E[N]$$

Yes, because X and N are assumed independent.

- What is this expectation product equal to?

0 because N has zero mean.

Derivation

Replace B and re-arrange loss:

$$\min_{H} \; E[\|(1+HC)X - HN\|^2]$$

Expand the squares:

$$\min_{H} \; \|1-HC\|^2 E[\|X\|^2] - 2H(1-HC)E[XN] + \|H\|^2 E[\|N\|^2]$$

cross-term is 0

Simplify:

$$\min_{H} \; \|1-HC\|^2 E[\|X\|^2] + \|H\|^2 E[\|N\|^2]$$

How do we solve this optimization problem?

Derivation

Differentiate loss with respect to H, set to zero, and solve for H:

$$\frac{\partial \text{loss}}{\partial H} = 0 = \frac{\partial \left[\ \|1 - HC\|^2 E[\|X\|^2]\ + \|H\|^2 E[\|N\|^2]\ \right]}{\partial H}$$

$$\Rightarrow -2C(1 - HC)E[\|X\|^2] + 2HE[\|N\|^2] = 0$$

$$\Rightarrow H = \frac{CE[\|X\|^2]}{C^2 E[\|X\|^2] + E[\|N\|^2]}$$

Derivation

$$H = \frac{CE[\|X\|^2]}{C^2 E[\|X\|^2] + E[\|N\|^2]}$$

Divide both numerator and denominator with $E[\|X\|^2]$

$$H = \frac{C}{C^2 + E[\|N\|^2]/E[\|X\|^2]}$$

Extract factor $1/\mathrm{C}$ and done!

$$H = \frac{C^2}{C^2 + E[\|N\|^2]/E[\|X\|^2]} \; \frac{1}{C}$$

Wiener Deconvolution

Apply inverse kernel and do not divide by zero:

$$x_{est} = F^{-1}\left(\frac{|F(c)|^2}{|F(c)|^2 + 1/SNR(\omega)} \cdot \frac{F(b)}{F(c)} \right)$$

noise-dependent damping factor

- Derived as solution to maximum-likelihood problem under Gaussian noise assumption
- Requires estimate of signal-to-noise ratio at each frequency

$$SNR(\omega) = \frac{\text{Expected signal squared at } \omega}{\text{Zero-mean noise variance at } \omega}$$

Wiener Deconvolution

Apply inverse kernel and do not divide by zero:

$$x_{est} = F^{-1}\left(\frac{|F(c)|^2}{|F(c)|^2 + \boxed{1/SNR(\omega)}} \cdot \frac{F(b)}{F(c)} \right)$$

noise-dependent damping factor

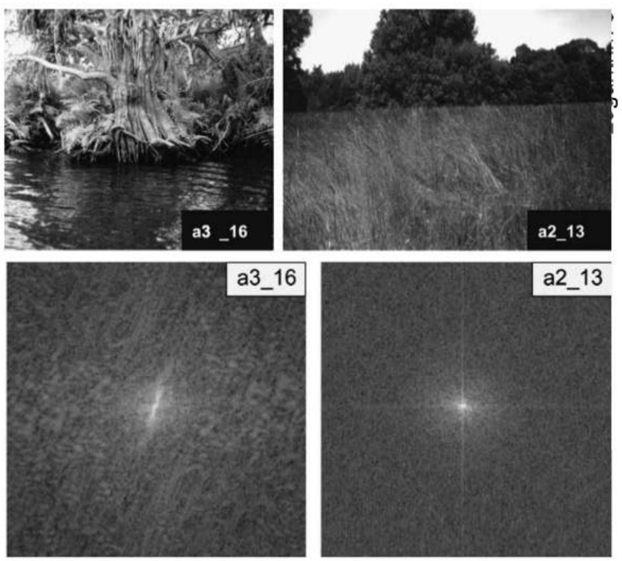
- Derived as solution to maximum-likelihood problem under Gaussian noise assumption
- Requires estimate of signal-to-noise ratio at each frequency

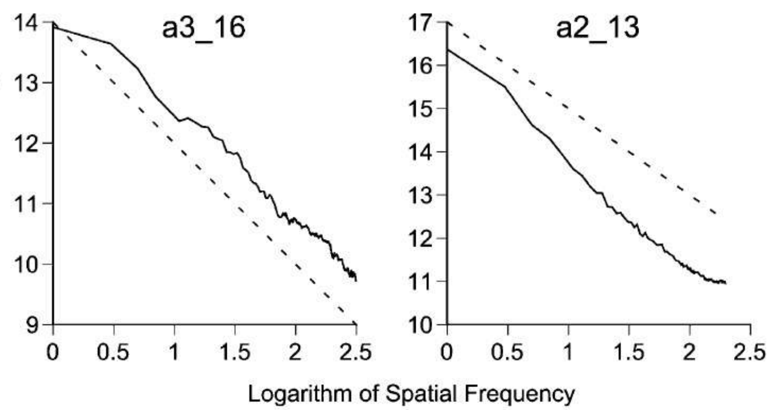
$$SNR(\omega) = \frac{\textbf{Expected signal squared at } \boldsymbol{\omega}}{\text{Zero-mean noise variance at } \omega}$$

Natural image and noise spectra

Natural images *tend* to have spectrum that scales with angular frequency as $1 / \omega^a$

- This is a *natural image statistic*





http://www.cnbc.cmu.edu/cns/papers/Balboa_PowerSpectra2003.pdf

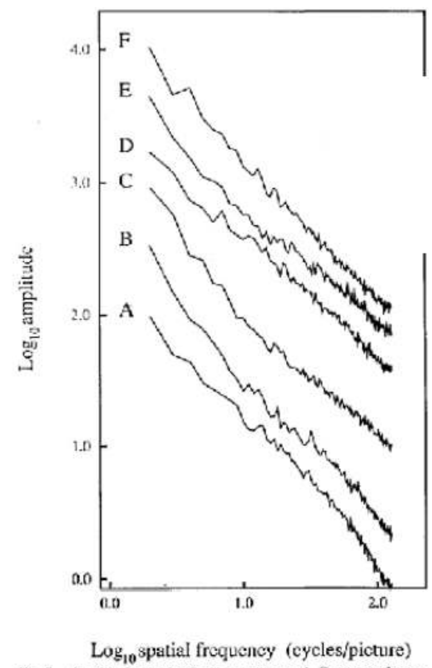
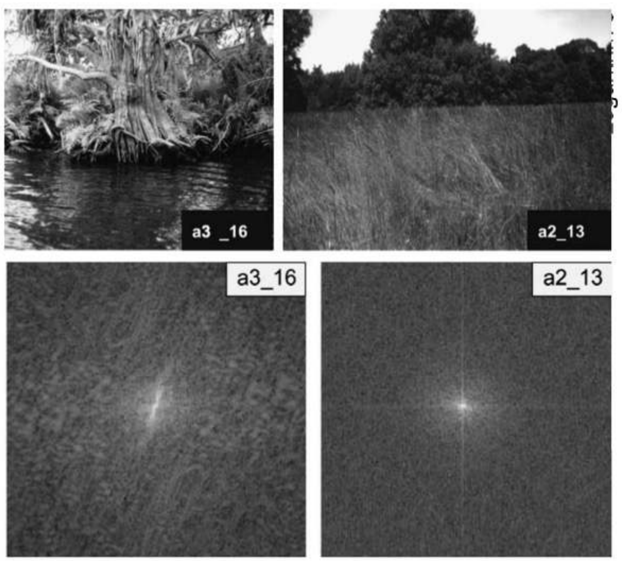


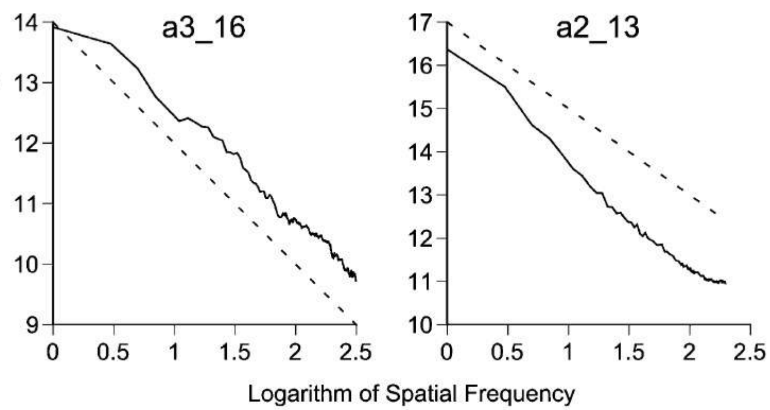
Fig. 8. Amplitude spectra for the six images A–F, averaged across all orientations. The spectra have been shifted up for clarity. On these log–log coordinates the spectra fall off by a factor of roughly $1/f$ (a slope of -1). Therefore the power spectra fall off as $1/f^2$.

Natural image and noise spectra

Natural images *tend* to have spectrum that scales with angular frequency as $1 / \omega^2$

- This is a *natural image statistic*





http://www.cnbc.cmu.edu/cns/papers/Balboa_PowerSpectra2003.pdf

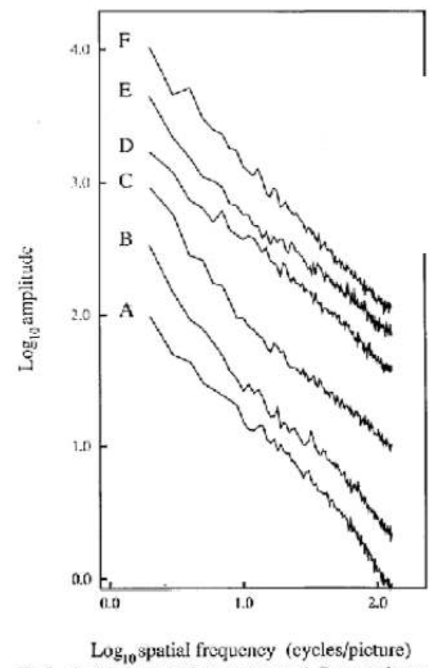


Fig. 8. Amplitude spectra for the six images A–F, averaged acros all orientations. The spectra have been shifted up for clarity. O these log–log coordinates the spectra fall off by a factor of roughl $1/f$ (a slope of −1). Therefore the power spectra fall off as $1/f^2$.

Noise tends to have flat spectrum, $\sigma(\omega)$ = constant

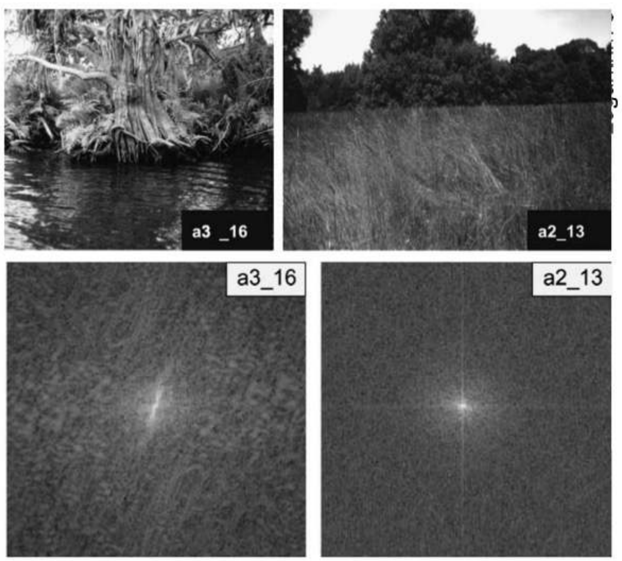
- We call this white noise

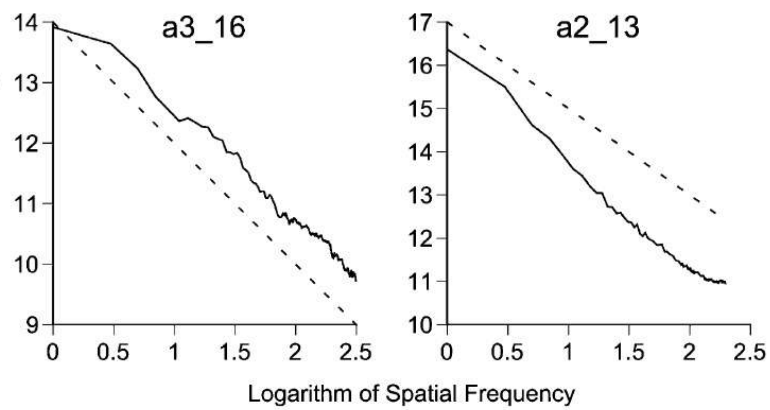
What is the SNR?

Natural image and noise spectra

Natural images *tend* to have spectrum that scales with angular frequency as $1 / \omega^2$

- This is a *natural image statistic*





http://www.cnbc.cmu.edu/cns/papers/Balboa_PowerSpectra2003.pdf

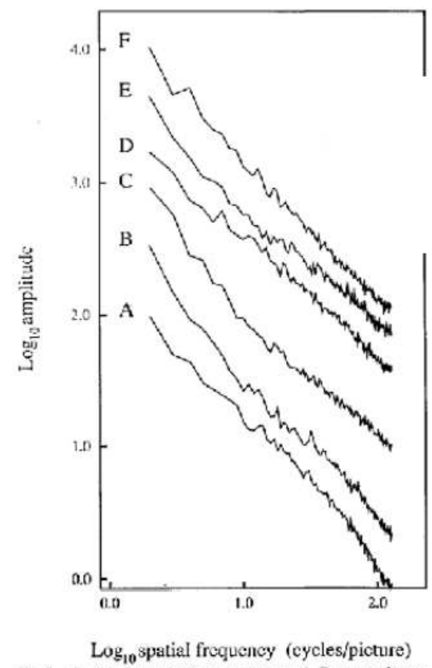


Fig. 8. Amplitude spectra for the six images A–F, averaged acros all orientations. The spectra have been shifted up for clarity. On these log-log coordinates the spectra fall off by a factor of roughly 1/f (a slope of −1). Therefore the power spectra fall off as $1/f^2$.

Noise tends to have flat spectrum, $\sigma(\omega)$ = constant

- We call this white noise

Therefore, we have that: SNR(ω) is proportional to $1 / \omega^2$

Wiener Deconvolution

Apply inverse kernel and do not divide by zero:

$$x_{est} = F^{-1}\left(\frac{|F(c)|^2}{|F(c)|^2 + \boxed{1/SNR(\omega)}} \cdot \frac{F(b)}{F(c)}\right)$$

noise-dependent damping factor

- Derived as solution to maximum-likelihood problem under Gaussian noise assumption
- Requires estimate of signal-to-noise ratio at each frequency

$$SNR(\omega) = \frac{\text{Expected signal squared at } \omega}{\text{Zero-mean noise variance at } \omega}$$

Wiener Deconvolution

Apply inverse kernel and do not divide by zero:

$$x_{est} = F^{-1}\left(\frac{|F(c)|^2}{|F(c)|^2 + \alpha\omega^2} \cdot \frac{F(b)}{F(c)}\right)$$

amplitude-dependent damping factor

- Derived as solution to maximum-likelihood problem under Gaussian noise assumption
- Requires noise of signal-to-noise ratio at each frequency

$$\mathrm{SNR}(\omega) = \frac{1}{\alpha\,\omega^2}$$

Wiener Deconvolution

Apply inverse kernel and do not divide by zero:

$$x_{est} = F^{-1} \left(\frac{|F(c)|^2}{|F(c)|^2 + \omega^2} \cdot \frac{F(b)}{F(c)} \right)$$

amplitude-dependent damping factor

- Derived as solution to maximum-likelihood problem under Gaussian noise assumption
- Requires noise of signal-to-noise ratio at each frequency

$$\mathrm{SNR}(\omega) = \frac{1}{\omega^2}$$

Question

Can we undo the compression blur in JPEG this way?

YES

No

Question

Can we undo lens blur by simply deconvolving a JPEG image?

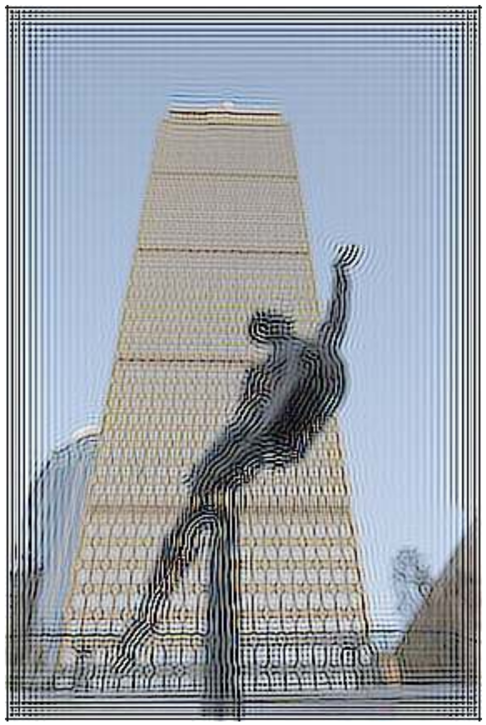
- All the blur processes we discuss today happen *optically* (before capture by the sensor).
- The blurring that occurs due to compression cannot be described with a single blur kernel
- **We could try to remove optical blur though from a JPEG image…**

But, are JPEG images linear?

- No, because of gamma encoding.
- Before deblurring, you must linearize your images.

The importance of linearity


blurry input


deconvolution without linearization


deconvolution after linearization


original

Regularized Deconvolution

$$\min_x ||b - c * x||^2 + ||\nabla x||^2$$

gradient *regularization*

Regularized Deconvolution Derivation

$$\min_{x} \left[\|b - c * x\|^2 + \|\nabla x\|^2\right]$$

$$\updownarrow$$

$$\min_{X} \left[\|B - CX\|^2 + \|i\omega X\|^2\right]$$

Not minus because it is the norm of a complex number

Expand the squares:

$$\min_{X} \|B\|^2 - 2CBX + C^2\|X\|^2 + \omega^2 \|X\|^2$$

Differentiate loss with respect to X, set to zero, and solve for X:

$$\frac{\partial \text{loss}}{\partial X} = 0 = \frac{\partial \left[\|B\|^2 - 2CBX + C^2\|X\|^2 + \omega^2 \|X\|^2\right]}{\partial X}$$

Regularized Deconvolution Derivation

Differentiate loss with respect to X, set to zero, and solve for X:

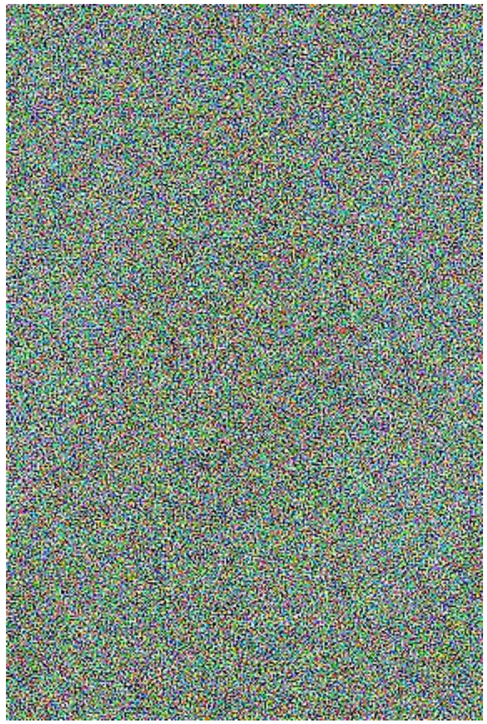
$$\frac{\partial \text{loss}}{\partial X} = 0 = \frac{\partial \; [\; \|B\|^2 - 2CBX + C^2\|X\|^2 + \omega^2 \; \|X\|^2]}{\partial X}$$

$$\Rightarrow -2CB + 2C^2X + 2\omega^2X = 0$$

$$\Rightarrow X = \frac{2CB}{2C^2 + 2\omega^2} = \frac{C^2}{C^2 + \omega^2}\frac{B}{C}$$

Deconvolution comparisons

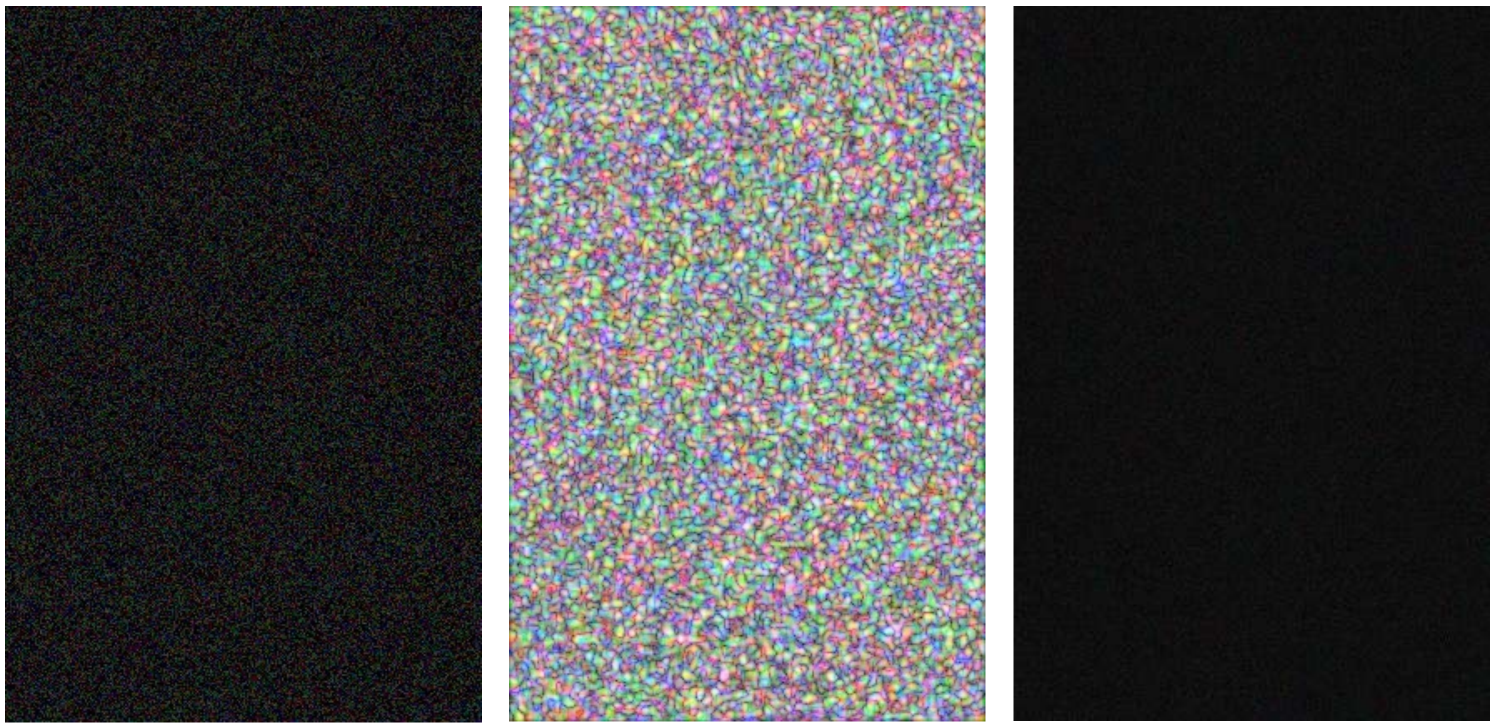

blurry input


naive deconvolution


gradient regularization


original

… and a proof-of-concept demonstration



noisy input naive deconvolution gradient regularization

Regularized Deconvolution Derivation

Differentiate loss with respect to X, set to zero, and solve for X:

$$\frac{\partial \text{loss}}{\partial X} = 0 = \frac{\partial\ [\ \|B\|^2 - 2CBX + C^2\|X\|^2 + \omega^2\ \|X\|^2]}{\partial X}$$

$$\Rightarrow -2CB + 2C^2X + 2\omega^2X = 0$$

$$\Rightarrow X = \frac{2CB}{2C^2 + 2\omega^2} = \frac{C^2}{C^2 + \omega^2}\frac{B}{C}$$

Regularized Deconvolution Solution

For natural images and white noise, equivalent to Weiner deconvolution:

$$X_{est} = F^{-1}\left(\frac{|F(c)|^2}{|F(c)|^2 + \omega^2} \cdot \frac{F(b)}{F(c)}\right)$$

$$X = \frac{2CB}{2C^2 + 2\omega^2} = \frac{C^2}{C^2 + \omega^2}\frac{B}{C}$$

Can we do better than that?

Can we do better than that?

Use different gradient regularizations:

- L_2 gradient regularization (Tikhonov regularization, same as Wiener deconvolution)

$$\min_x ||b - c * x||^2 + ||\nabla x||^2$$

- L_1 gradient regularization (sparsity regularization, same as total variation)

$$\min_x ||b - c * x||^2 + ||\nabla x||^1$$

How do we solve for these?

- $L_{n<1}$ gradient regularization (fractional regularization)

$$\min_x ||b - c * x||^2 + ||\nabla x||^{0.8}$$

All of these are motivated by natural image statistics. Active research area.

Comparison of gradient regularizations

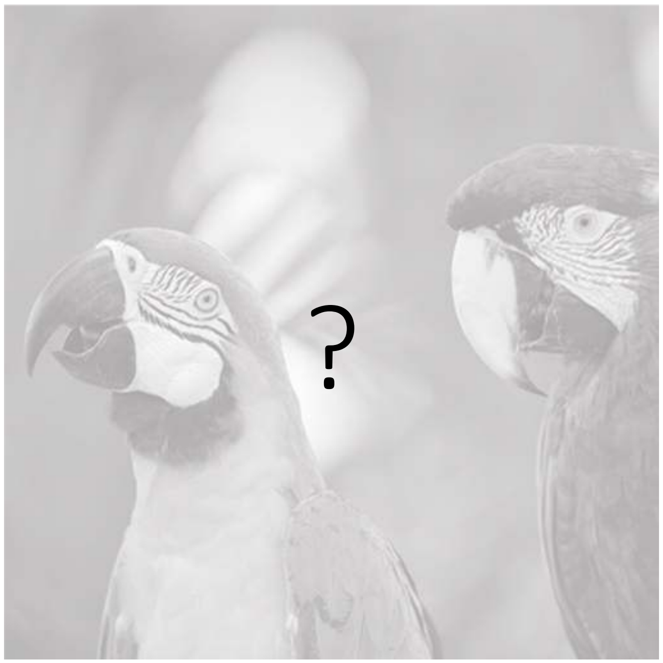
input

squared gradient regularization

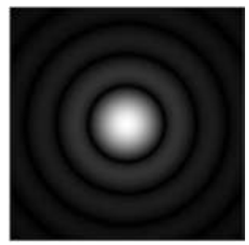
fractional gradient regularization

Deconvolution

If we know b and c, can we recover x?



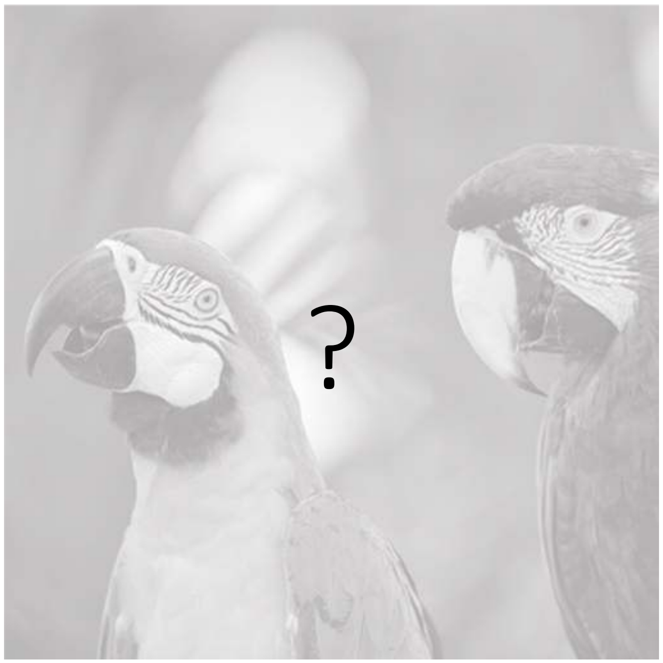
How do we measure this?

$*$  $=$ 

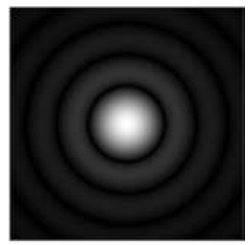
$$x \quad * \quad c \quad = \quad b$$

Deconvolution

If we know b and c, can we recover x?



$*$

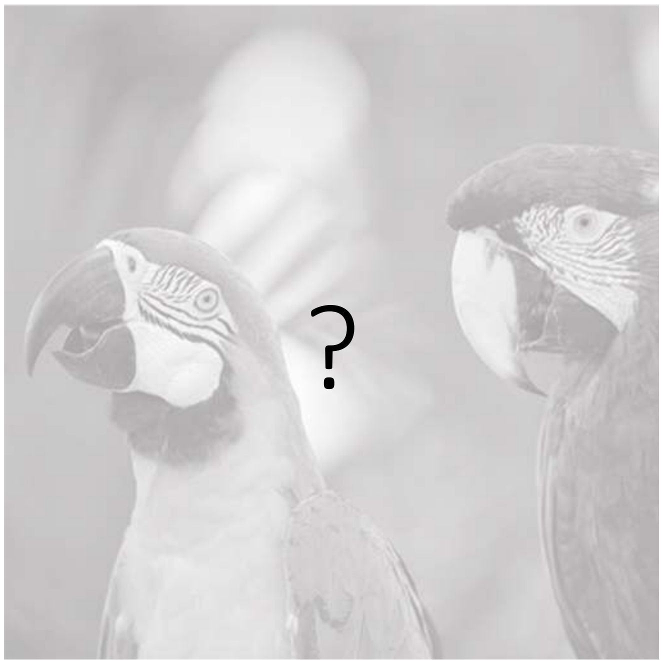


$=$

$$x \quad * \quad c \quad = \quad b$$

Blind deconvolution

If we know b, can we recover x and c?



$*$

$=$

$$x \quad * \quad c \quad = \quad b$$

Camera shake

Removing Camera Shake from a Single Photograph

Rob Fergus[1] Barun Singh[1] Aaron Hertzmann[2] Sam T. Roweis[2] William T. Freeman[1]

[1]MIT CSAIL [2]University of Toronto

Figure 1: *Left*: An image spoiled by camera shake. *Middle*: result from Photoshop “unsharp mask”. *Right*: result from our algorithm.

Camera shake as a filter

If we know b, can we recover x and c?

image from static camera

$*$

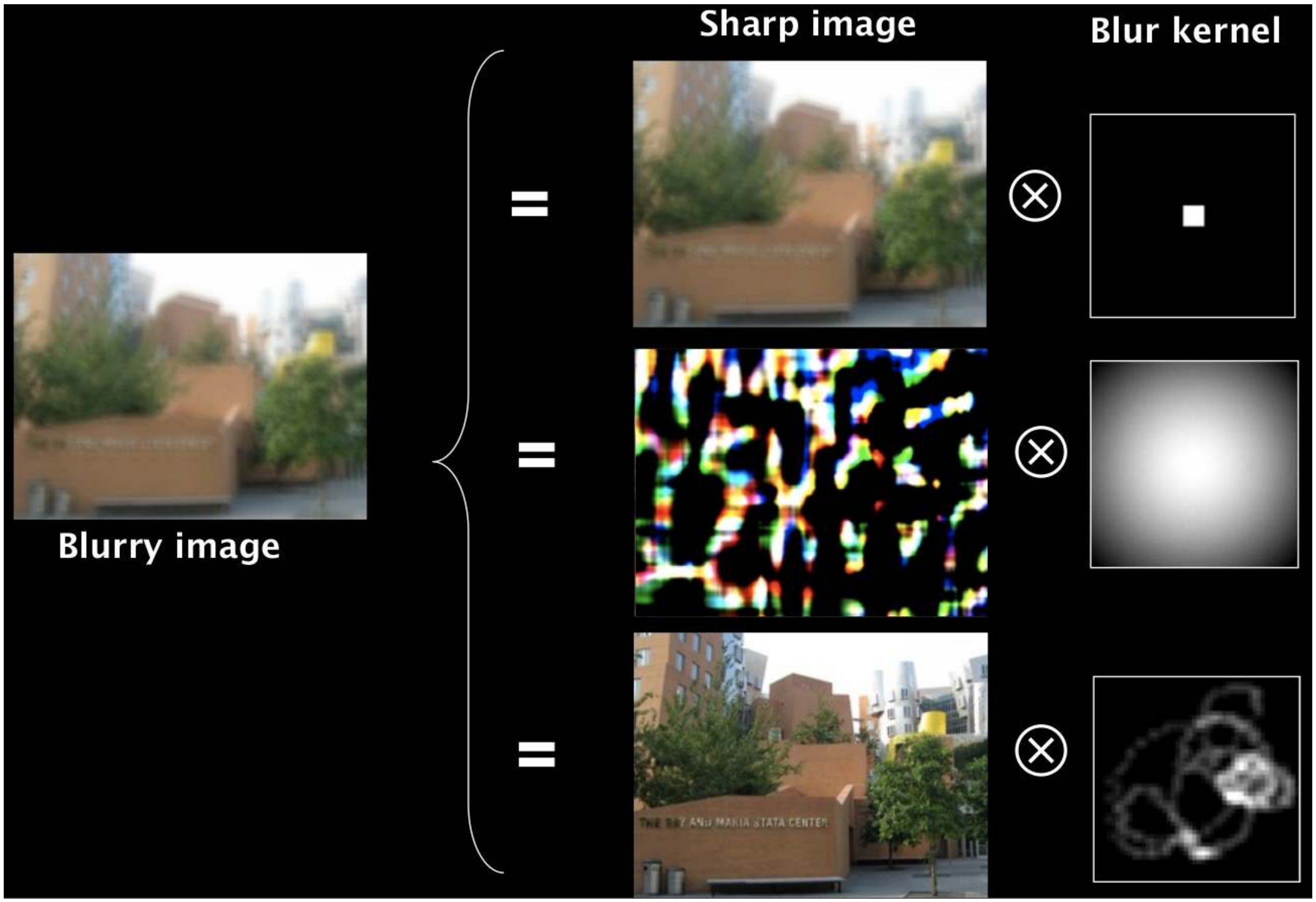
PSF from camera motion

$=$

image from shaky camera

$$x \quad * \quad c \quad = \quad b$$

Multiple possible solutions



How do we detect this one?

Use prior information

Among all the possible pairs of images and blur kernels, select the ones where:

- The image “looks like” a natural image.
- The kernel “looks like” a motion PSF.

Use prior information

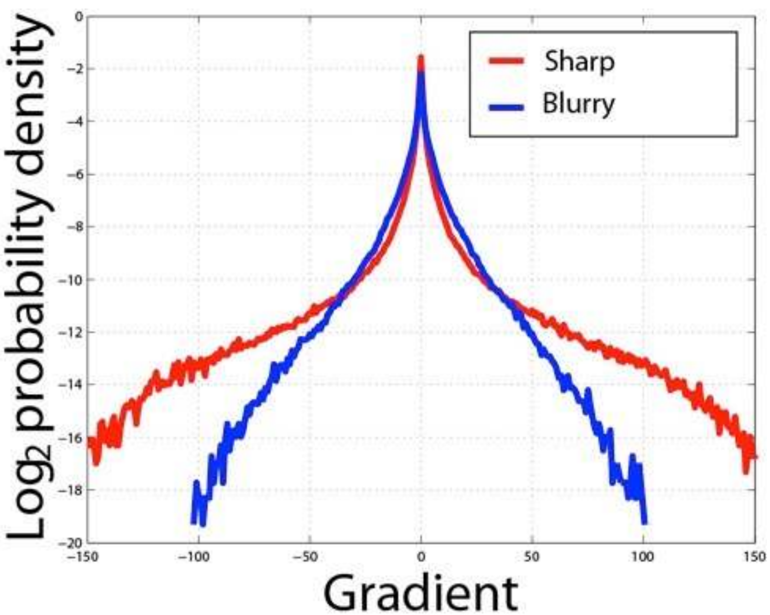
Among all the possible pairs of images and blur kernels, select the ones where:

- The image “looks like” a natural image.

- The kernel “looks like” a motion PSF.

Shake kernel statistics

Gradients in natural images follow a characteristic "heavy-tail" distribution.

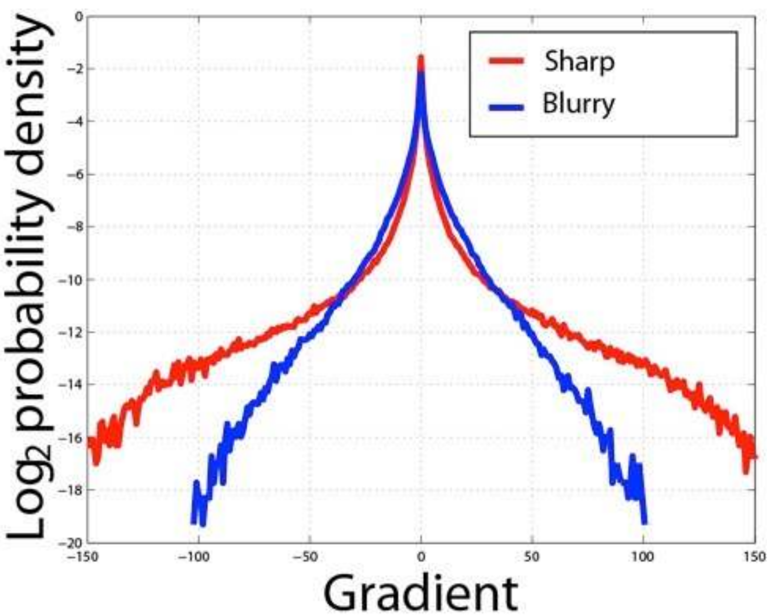


sharp natural image

blurry natural image

Shake kernel statistics

Gradients in natural images follow a characteristic "heavy-tail" distribution.



Can be approximated by $\| \nabla x \|^{0.8}$

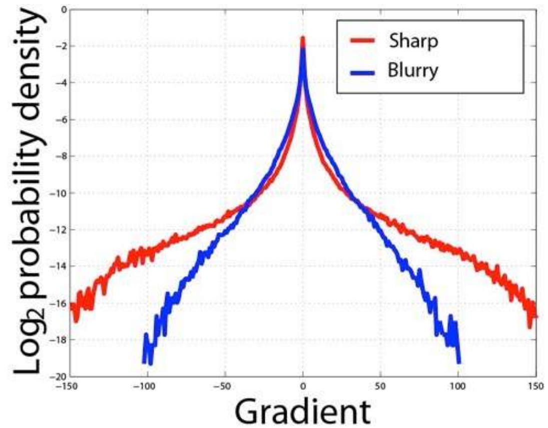
sharp natural image

blurry natural image

Use prior information

Among all the possible pairs of images and blur kernels, select the ones where:

- The image “looks like” a natural image.



Gradients in natural images follow a characteristic “heavy-tail” distribution.

- The kernel “looks like” a motion PSF.

Shake kernels are very sparse, have continuous contours, and are always positive

How do we use this information for blind deconvolution?

Regularized blind deconvolution

Solve regularized least-squares optimization

$$\min_{x,c} ||b - c * x||^2 + ||\nabla x||^{0.8} + ||c||^1$$

What does each term in this summation correspond to?

Regularized blind deconvolution

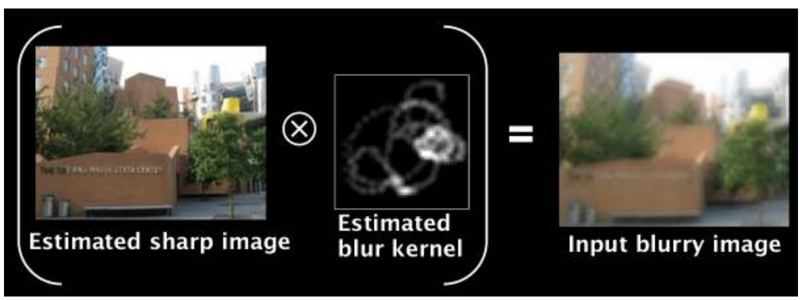
Solve regularized least-squares optimization

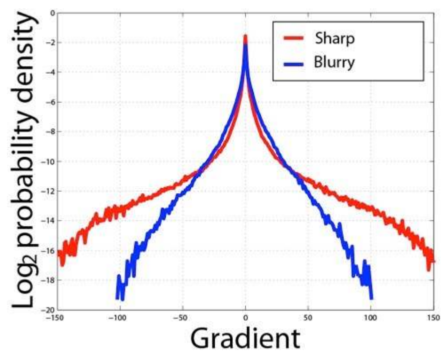
$$\min_{x,c} ||b - c * x||^2 + ||\nabla x||^{0.8} + ||c||^1$$

data term

natural image prior

shake kernel prior



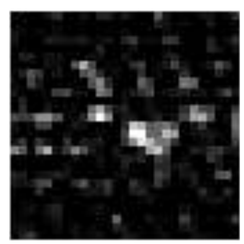


Note: Solving such optimization problems is complicated (no longer *linear* least squares).

A demonstration

input

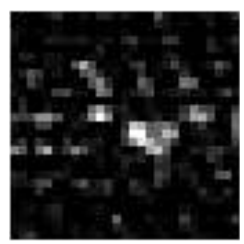
deconvolved image and kernel



A demonstration

input

deconvolved image and kernel



This image looks worse than the original…

This doesn’t look like a plausible shake kernel…

Regularized blind deconvolution

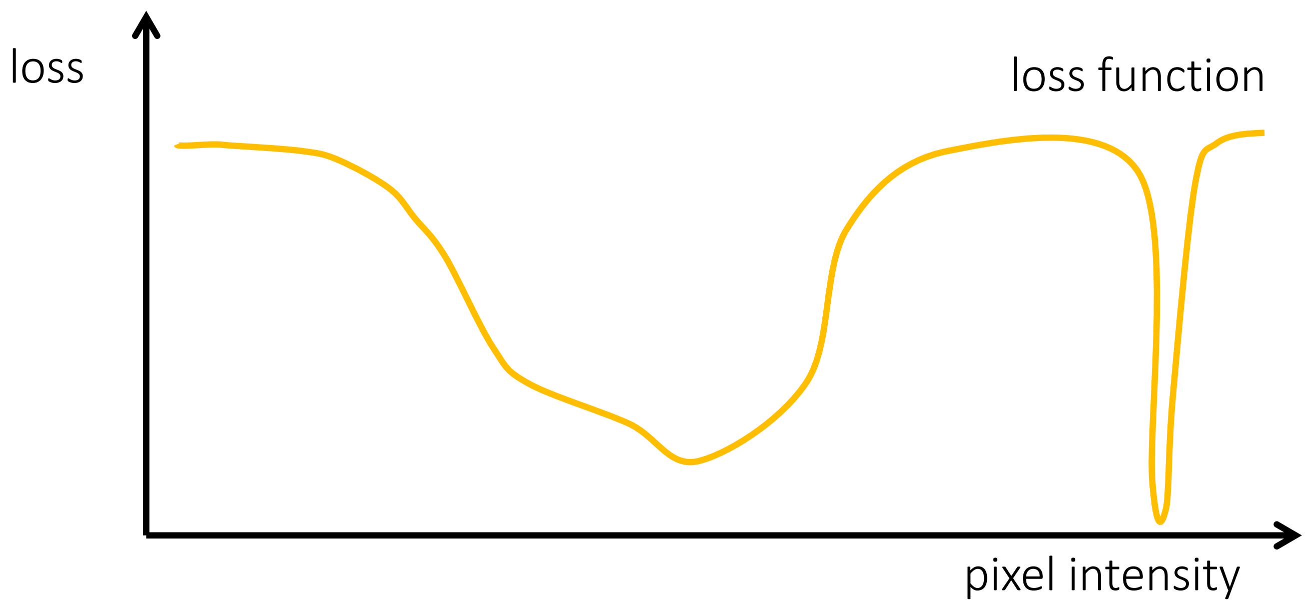
Solve regularized least-squares optimization

$$\min_{x,c} \underbrace{||b - c * x||^2 + ||\nabla x||^{0.8} + ||c||^1}_{\text{loss function}}$$

Regularized blind deconvolution

Solve regularized least-squares optimization

$$\min_{x,b} \underbrace{||b - c * x||^2 + ||\nabla x||^{0.8} + ||c||^1}_{\text{loss function}}$$

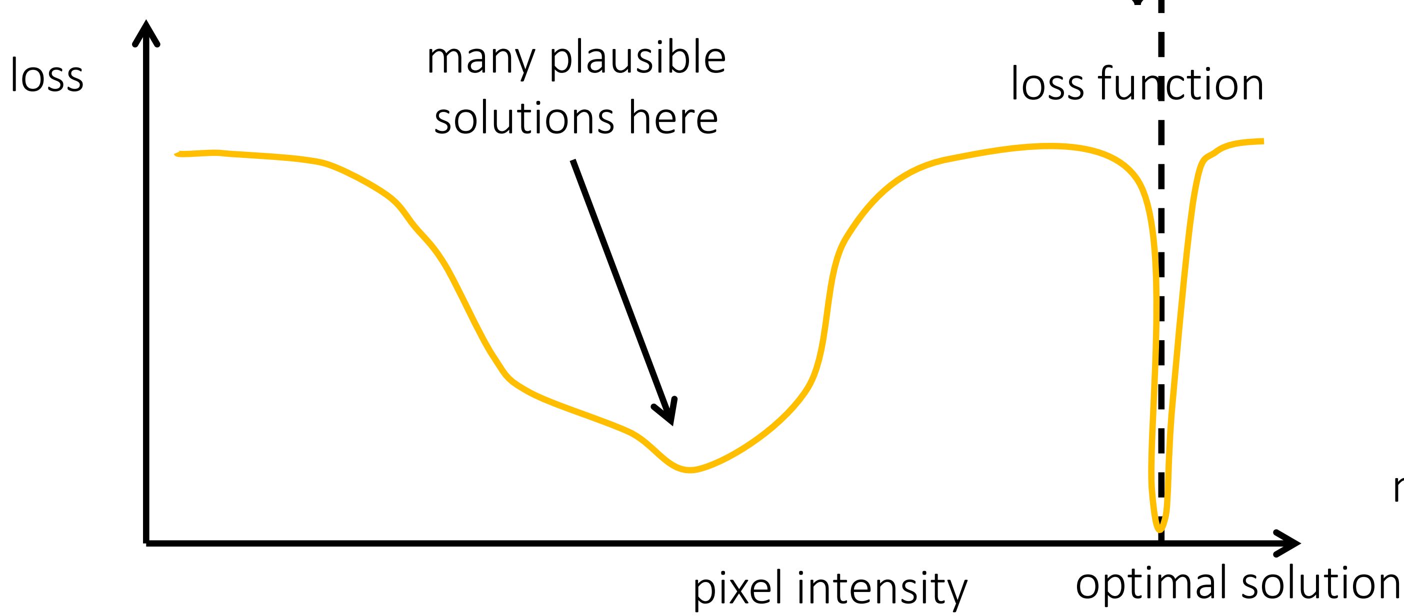


Where in this graph is the solution we find?

Regularized blind deconvolution

Solve regularized least-squares optimization

$$\min_{x,b} \; \|b - c * x\|^2 + \|\nabla x\|^{0.8} + \|c\|_1$$

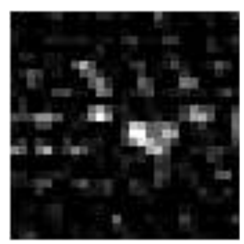


Rather than keep just minimum, do a weighted average of all solutions

A demonstration

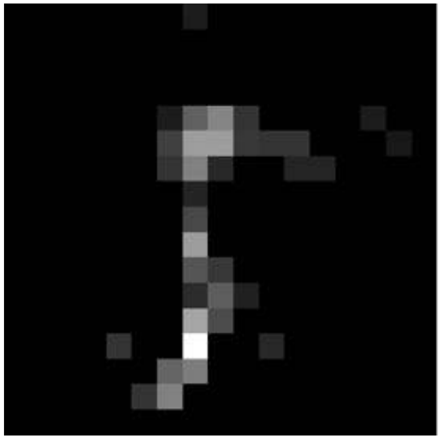
input

maximum-only



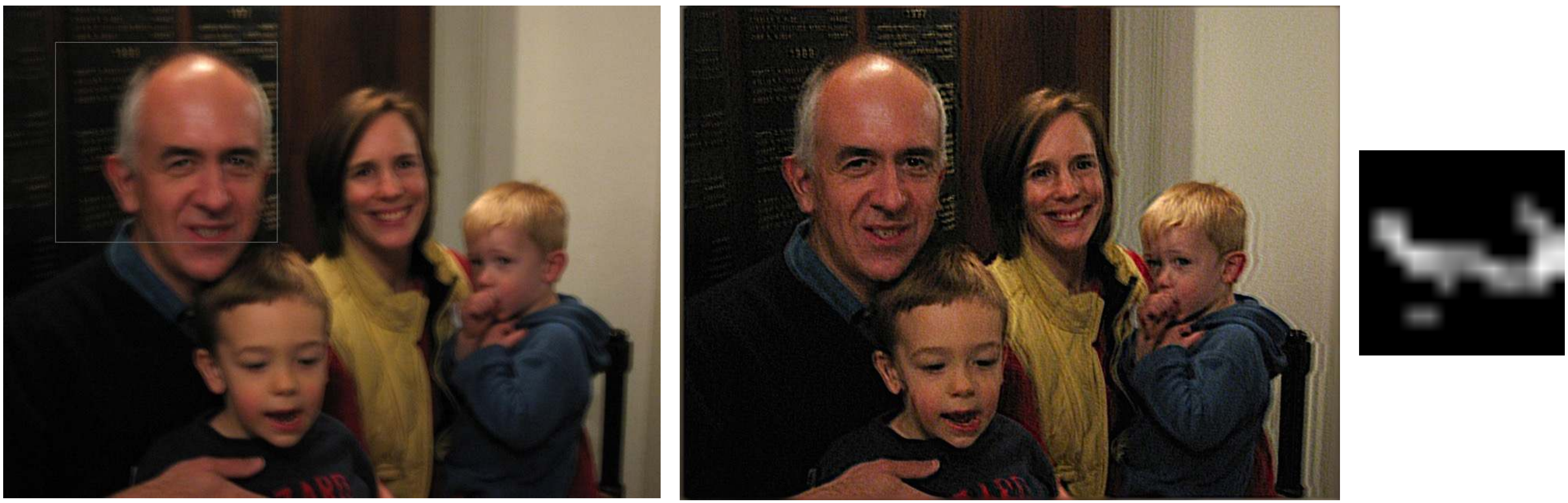
average

Results on real shaky images

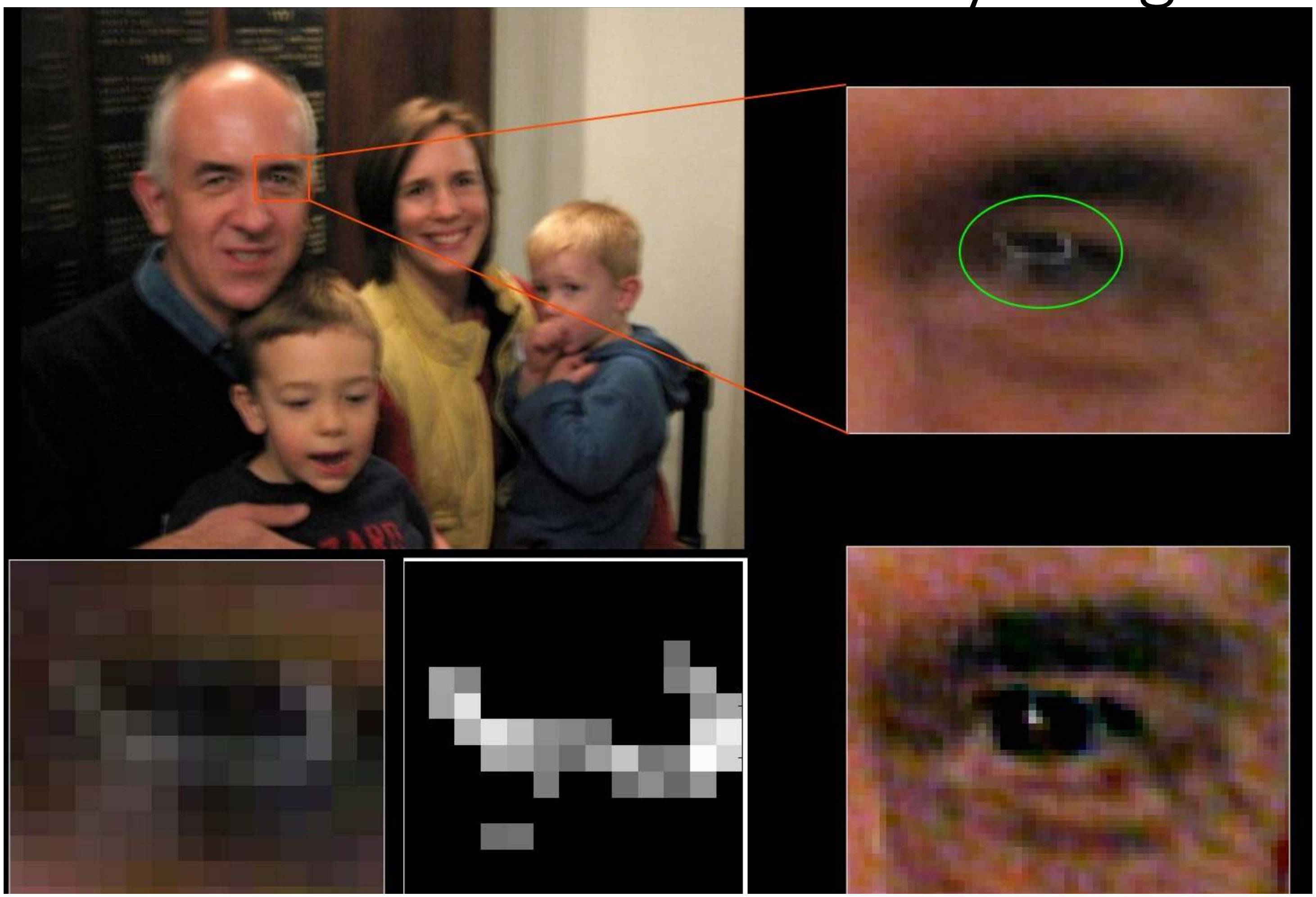


Results on real shaky images

Results on real shaky images



Results on real shaky images



Why are our images blurry?

- Lens imperfections.
- Camera shake.
- Scene motion.
- Depth defocus.

Can we solve all of these problems using (blind) deconvolution?

Why are our images blurry?

- Lens imperfections.
- Camera shake.
- Scene motion.
- Depth defocus.

Can we solve all of these problems using (blind) deconvolution?

- We can deal with (some) lens imperfections and camera shake, because their blur is shift invariant.
- We cannot deal with scene motion and depth defocus, because their blur is not shift invariant.
- See next lecture on coded photography….

Regularized Inversion

Deconvolution

$$\min_x ||b - c * x||^2 + ||\nabla x||^2$$

General linear inverse problem

$$\min_x ||b - Fx||^2 + ||\nabla x||^2$$

What problem are we solving if F is the identify matrix and x is an image?

Regularized Inversion

Regularized linear inverse problem

$$\min_x \; [\|b - Fx\|^2 + \|\nabla x\|^2]$$

Write the derivative as finite differences. What does G look like?

$$\min_x \; [\|b - Fx\|^2 + \|\mathrm{G}x\|^2]$$

Expand

$$\min_x \; [(b - Fx)^T(b - Fx) + x^T G^T \mathrm{G}x]$$

Solve for x, by setting the derivative equal to 0

Regularized Inversion

Use different **gradient regularizations**:

- L_2 gradient regularization (Tikhonov regularization, same as Wiener deconvolution)

$$\min_x ||b - Fx||^2 + ||\nabla x||^2$$

- L_1 gradient regularization (sparsity regularization, same as total variation)

$$\min_x ||b - Fx||^2 + ||\nabla x||^1$$

- $L_{n<1}$ gradient regularization (fractional regularization)

$$\min_x ||b - Fx||^2 + ||\nabla x||^{0.8}$$

All of these are motivated by natural image statistics. Active research area.

Questions?